

u - małe:



$$i' = H[i]$$

u - duże

Niech $0, \dots, m-1$ indeksy H. Chcemy mieć funkcję

$$h: U \rightarrow \{0, \dots, m-1\}$$

Problem - h może nie być różnowartościowe.

Funkcje hasujące

$$\forall k: 0, \dots, m-1$$

$$\sum_x P(h(x) = k) = \frac{1}{m}$$

Przykłady funkcji hasujących

• $h(k) = k \bmod m$ (m - liczba pierzasa, w przypadku $k \bmod 2^l$, mamy l ostatnich bitów k - te bity mogą być rozdzielone równomiernie)

• $h(k) = \lfloor m(kA - \lfloor kA \rfloor) \rfloor$
 $A \in [0, 1]$ → jedna z rekomendowanych wartości A jest: $\frac{\sqrt{5}-1}{2} \approx 0,66\dots$

$$h: U \rightarrow \{0, \dots, m-1\}$$

Pamiętanie elementów S



Spać mi się chce!

Problem - kolizje w końcu następu (paradoks urodzin).

Rozwiązanie - w tablicy trzymamy listy - jeśli następ kolizja - następny element na listę

FAKT Jeśli h „zachowuje się losowo” to średni koszt wyszukiwania klucza wynosi $O(1 + \frac{\alpha}{m})$, gdzie n - liczba kluczy w tablicy, m - rozmiar tablicy

$\frac{n}{m} \rightarrow$ współczynnik zapełnienia tablicy.

Adresowanie otwarte - szukamy wolnego miejsca w przypadku kolizji.

Używamy funkcji $h: \mathcal{U} \times \{0, \dots, m-1\} \rightarrow \{0, \dots, m-1\}$

t.j. $\left. \begin{matrix} h(k, 0) \\ \vdots \\ h(k, m-1) \end{matrix} \right\} \begin{matrix} \text{warunek:} \\ \text{te wartości} \\ \text{są permutacją} \\ \text{liczb } \{0, \dots, m-1\} \end{matrix}$

Metoda liniowa:

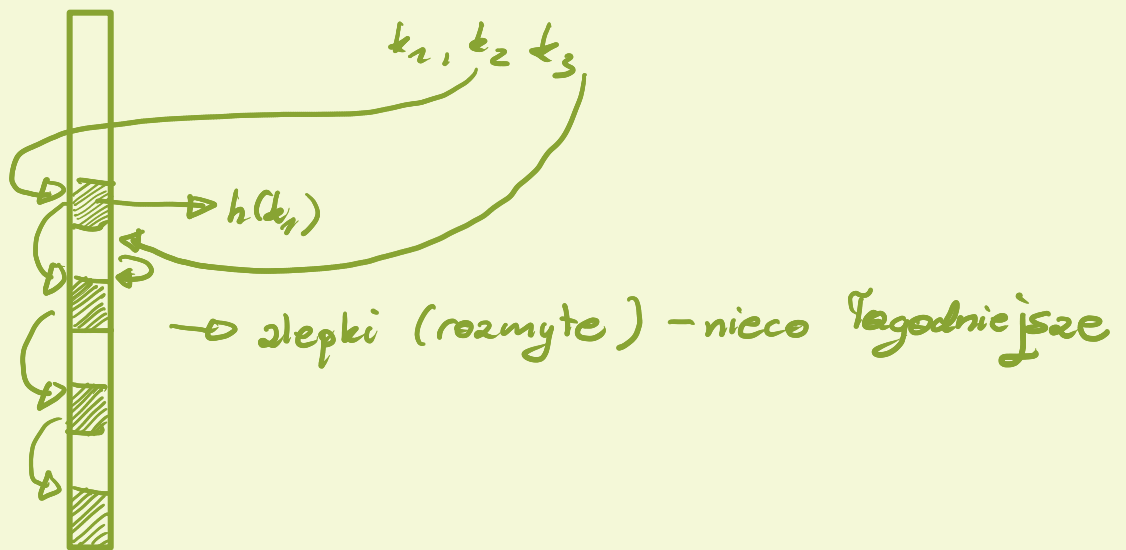


$\left\{ \begin{array}{l} h(k, i) = (h'(k) + i) \bmod m \\ \text{gdzie } h': \mathcal{U} \rightarrow \{0, \dots, m-1\} \\ \text{- funkcja haszująca} \end{array} \right.$

Metoda kwadratowa

$$h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$$

$c_1, c_2 \leftarrow \text{stałe}$



Podwójne haszowanie

$$h(k_i) = (h_1(k) + i h_2(k)) \bmod n$$

metoda liniowa - m permutacji pozycji (przesunięcia cykliczne)

$$\forall j \quad j, j+1, \dots, m-1, 0, \dots, j-1$$

metoda kwadratowa - też m permutacji

metoda podwójna - m^2 -permutacji

idealnie - $n!$ permutacji

Analiza kosztów przy założeniu

(*) każda permutacja jest jednakowo prawdopodobna, jako ciąg kontrolny (ciąg proponowanych pozycji)

FAKT - przy założeniu (*) i $\alpha = \frac{n}{m} < 1$ ocalkowane liczbę prób w operacji Search zebranych fiaskiem jest $\leq \frac{1}{1-\alpha}$

d-d:
Niech φ_i - prawdopodobieństwo, że wykonamy dokładnie i prób

Oczekiwana liczba prób

$$= 1 + \sum_{i=0}^{\infty} i p_i = (1)$$

Niech q_i - prawdopodobieństwo, że wykonamy co najmniej i prób. Wtedy $p_i = q_i - q_{i+1}$

$$(1) = 1 + (q_1 - q_2) + 2(q_2 - q_3) + 3(q_3 - q_4) \dots =$$

$$= 1 + q_1 + q_2 + q_3 + \dots = (2)$$

Ale $q_1 = \frac{n}{m} \alpha$, bo wykonamy co najmniej 1 próbę

$$q_2 = \frac{1}{m} \cdot \frac{n-1}{m-1} \leq \alpha^2$$

$$\vdots$$
$$q_i \leq \frac{1}{m} \cdot \frac{n-1}{m-1} \cdot \dots \cdot \frac{n-i+1}{m-i+1} \leq \alpha^i$$

Stąd

$$(2) \leq \sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha} \quad \blacksquare$$

$$\frac{n}{m} \geq \frac{n-1}{m-1}$$

$$nm - n \geq nm - m$$
$$m \leq n$$